

Python Coding for AI

Ch1: Python의 직관적 이해

Ch2: Python OOP

Ch3: What is Numpy & Why Numpy?

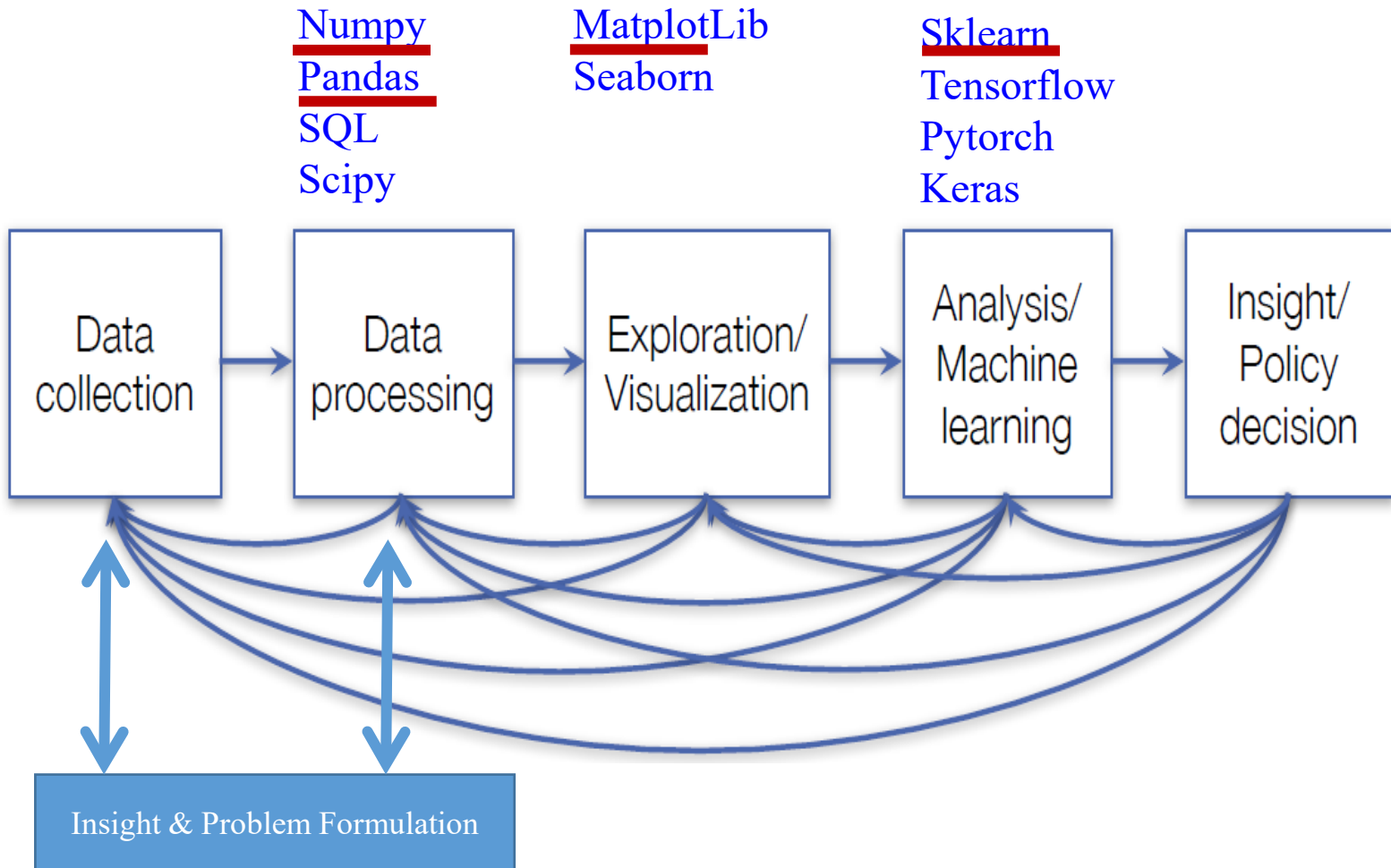
Ch4: What is Pandas & Why Pandas?

Ch5: What is Matplotlib & Why Matplotlib?

Ch6: What is Sklearn & Why Sklearn?

Ch7: What is Data Preprocessing & Why Data Preprocessing?

Major Steps of Data Analytics



Data for Machine Learning and Data Mining [1/3]

- **Boston Housing Data:** 1970년대 Boston지역 주택관련 여러 속성과 집값 데이터
- 506개 records, 14개 column (13개 attribute + 집 값)
 - **CRIM:** 범죄율
 - **INDUS:** 비소매상업지역 면적 비율
 - **NOX:** 일산화질소 농도
 - **RM:** 주택당 방 수
 - **LSTAT:** 인구 중 하위 계층 비율
 - **B:** 인구 중 흑인 비율
 - **PTRATIO:** 학생/교사 비율
 - **ZN:** 25,000 평방피트 초과 거주지역 비율
 - **CHAS:** 찰스강 경계에 위치한 경우 1, 아니면 0
 - **AGE:** 1940년 이전 건축된 주택 비율
 - **RAD:** 방사형 고속도로까지의 거리
 - **DIS:** 직업 센터의 거리
 - **TAX:** 재산세율
 - **MEDV:** 주택 가격의 median(단위: \$1,000)

14개 column

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33	36.2



데이터분석: 특정 집이 특정가격에 매물로 나왔을때 가격의 적정성을 판단

Data for Machine Learning and Data Mining [2/3]

Energy Company Data

Company	Fixed_charge	RoR	Cost	Load	Δ Demand	Sales	Nuclear	Fuel_Cost
Arizona	1.06	9.2	151	54.4	1.6	9077	0	0.628
Boston	0.89	10.3	202	57.9	2.2	5088	25.3	1.555
Central	1.43	15.4	113	53	3.4	9212	0	1.058
Commonwealth	1.02	11.2	168	56	0.3	6423	34.3	0.7
Con Ed NY	1.49	8.8	192	51.2	1	3300	15.6	2.044
Florida	1.32	13.5	111	60	-2.2	11127	22.5	1.241
Hawaiian	1.22	12.2	175	67.6	2.2	7642	0	1.652
Idaho	1.1	9.2	245	57	3.3	13082	0	0.309
Kentucky	1.34	13	168	60.4	7.2	8406	0	0.862
Madison	1.12	12.4	197	53	2.7	6455	39.2	0.623
Nevada	0.75	7.5	173	51.5	6.5	17441	0	0.768
New England	1.13	10.9	178	62	3.7	6154	0	1.897
Northern	1.15	12.7	199	53.7	6.4	7179	50.2	0.527
Oklahoma	1.09	12	96	49.8	1.4	9673	0	0.588
Pacific	0.96	7.6	164	62.2	-0.1	6468	0.9	1.4
Puget	1.16	9.9	252	56	9.2	15991	0	0.62
San Diego	0.76	6.4	136	61.9	9	5714	8.3	1.92
Southern	1.05	12.6	150	56.7	2.7	10140	0	1.108
Texas	1.16	11.7	104	54	-2.1	13507	0	0.636
Wisconsin	1.2	11.8	148	59.9	3.5	7287	41.1	0.702
United	1.04	8.6	204	61	3.5	6650	0	2.116
Virginia	1.07	9.3	174	54.3	5.9	10093	26.6	1.306

데이터분석: 어떤 Energy Company가 효율적으로 business를 하는지를 판단

Data for Machine Learning and Data Mining [3/3]

■ Credit Card Fraud Data (284,807 lines, 31 attributes)

- 2013년 9월중 2일간 신용카드 거래 기록 (492 frauds / 284,807 transactions.)
- **Time**: 은 첫번째 거래로부터 현재 거래까지의 시간
- **Amount**: 거래금액
- **Class**: 1 (Fraud, 사기) 또는 0 (normal, 정상)
- 나머지 attribute는 정보는 비공개

31개 attributes

Time	V1	V2	V3	...	V28	Amount	Class
0	-1.35981	-0.07278	2.536347	...	-0.02105	149.62	0
0	1.191857	0.266151	0.16648	...	0.014724	2.69	0
1	-1.35835	-1.34016	1.773209	...	-0.05975	378.66	0
1	-0.96627	-0.18523	1.792993	...	0.061458	123.5	0
2	-1.15823	0.877737	1.548718	...	0.215153	69.99	0
2	-0.42597	0.960523	1.141109	...	0.08108	3.67	0
4	1.229658	0.141004	0.045371	...	0.005168	4.99	0



데이터분석: Credit Card 사용내용이 발생했을때 Fraud인지 아닌지를 예측

Python의 Built-In Data Type들만 가지고 아래 데이터들을 표현한다면 List (2D)를 사용할것입니다.

Simple Matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

당뇨병 환자 관련 데이터

Age	Sex	BMI	ABP	S1	S2	S3	S4	S5	S6	공복혈당
0.038	0.051	0.062	0.022	-0.044	-0.035	-0.043	-0.003	0.020	-0.018	151
-0.002	-0.045	-0.051	-0.026	-0.001	-0.019	0.074	-0.039	-0.068	-0.092	75
0.085	0.051	0.044	-0.006	-0.046	-0.034	-0.032	-0.003	0.002	-0.026	141
-0.089	-0.045	-0.012	-0.037	0.012	0.025	-0.036	0.034	0.023	-0.009	206

Basic Data Types vs

Collection Data Types

Remember?

- Int
 - $X = 3$
- Float
 - $X = 3.1$
- Boolean
 - $X = \text{True}$

- String
 - $X = \text{"SNU"}$
- List
 - $X = [3, 7, 2, 1]$
- Tuple
 - $X = (1, 7, 2, 4)$
- Set
 - $X = \{3, 6, 1, 2\}$
- Dictionary
 - $X = \{\text{"Korea"}: 3500, \text{"USA"}: 2000\}$

Two Dimensional Arrays for Matrix

B

	columns				
	0	1	2	3	4
0	3	18	43	49	65
1	14	30	32	53	75
2	9	28	38	50	73
3	10	24	37	58	62
4	7	19	40	46	66

rows

Synonym

Table

Matrix

2-Dimensional Array

Cell: 칸

B[1] →

14	30	32	53	75
----	----	----	----	----

B[3][2] → 37

$$B = \begin{matrix} & \text{columns} \\ \begin{matrix} \text{rows} \\ \begin{bmatrix} 3 & 18 & 43 & 49 & 65 \\ 14 & 30 & 32 & 53 & 75 \\ 9 & 28 & 38 & 50 & 73 \\ 10 & 24 & 37 & 58 & 62 \\ 7 & 19 & 40 & 46 & 66 \end{bmatrix} \end{matrix} \end{matrix}$$

2D Lists in Python for Matrix

```
[4] data = [ [1, 2, 3, 4],  
             [5, 6, 7, 8],  
             [9, 10, 11, 12]  
           ]
```

```
[5] data[0]
```

```
[1, 2, 3, 4]
```

```
[6] data[1][2]
```

```
7
```

```
 data[2][5]
```

```
 -----  
IndexError  
/tmp/ipython-input-3178599576.py in <  
----> 1 data[2][5]
```

```
IndexError: list index out of range
```

data[0]

[1, 2, 3, 4]

data[1]

[5, 6, 7, 8]

data[2]

[9, 10, 11, 12]

columns

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

rows

C language
2D Array

columns

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

rows

수학책의
Matrix

2D List Example in Python : Sum of Matrix Elements

columns → 4

```
>>> table = [ [ 1, 2, 3, 4 ],  
               [ 5, 6, 7, 8 ],  
               [ 9, 10, 11, 12 ]  
               ]
```

rows = 3

Find the sum of all elements in a 2D array

```
def sum_matrix(table):
```

```
    sum = 0
```

```
    for row in range(0, len(table)):
```

```
        for col in range(0, len(table[row])):
```

```
            sum = sum + table[row][col]
```

```
    return sum
```

Number of rows in the table

Number of columns in the given row of the table

Tracing the Nested Loop

Find the sum of all elements in a 2D array

```
def sum_matrix(table):
    sum = 0
    for row in range(0, len(table) ):
        for col in range(0, len(table[row]) ):
            sum = sum + table[row][col]
            print(row, col, sum)
    return sum
```

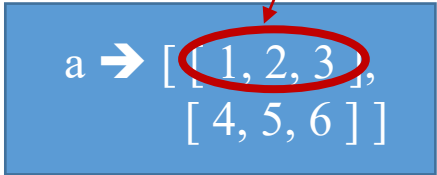
```
>>> table = [ [ 1, 2, 3, 4 ],
               [ 5, 6, 7, 8 ],
               [ 9, 10, 11, 12 ]
               ]
```

```
>>> sum_matrix(table)
```

row	col	sum
0	0	1
0	1	3
0	2	6
0	3	10
1	0	15
1	1	21
1	2	28
1	3	36
2	0	45
2	1	55
2	2	66
2	3	78

Row Access and Column Access in Matrix

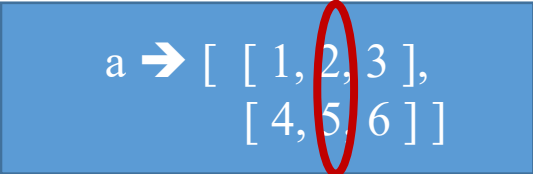
How to access 0th row ?



a → $\begin{bmatrix} 1, 2, 3 \\ 4, 5, 6 \end{bmatrix}$

```
a = [ [ 1, 2, 3], [4, 5, 6] ]  
row = 1  
rowList = a[row]  
print(rowList)
```

How to access 1st column ?



a → $\begin{bmatrix} 1, 2, 3 \\ 4, 5, 6 \end{bmatrix}$

```
a = [ [ 1, 2, 3], [4, 5, 6] ]  
col = 1  
colList = [ ]  
for i in range( len(a) ):  
    colList = colList + [ a[i][col] ]  
print( colList )
```



colList.append(a[i][col])

Python List로 표현된 Matrix처리의 불편함

```
data = [[1, 2, 3, 4],  
        [5, 6, 7, 8],  
        [9, 10, 11, 12]]
```

Statistics module

```
>>>from statistics import *  
>>>print( mean( [2, 6, 10] ) )
```

X_data Y_data

```
data = [[1, 2, 3, 4],  
        [5, 6, 7, 8],  
        [9, 10, 11, 12]]
```

Sklearn module

```
>>>from sklearn import linear_model  
>>>  
>>> regr = linear_model.LinearRegression( )  
>>> regr.fit( X_data, Y_data )  
>>> regr
```

matplotlib module

```
>>>from matplotlib.pyplot as plt  
>>>  
>>> plt.plot( X_data, Y_data )
```

Matrix Data를 List로 표현한다면!

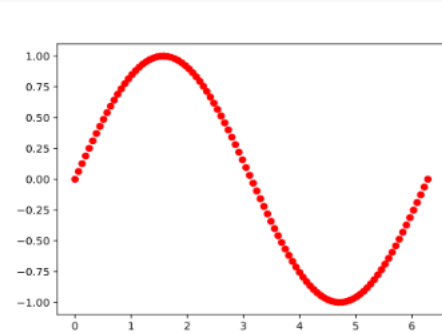
- List는 Python의 대표 Data Type 이라 Matrix데이터도 표현은 하지만....
- Matrix Data에 대한 다양한 Linear Algebra 작업을 한다면, Matrix Data를 List로 표현해서는 Coding이 복잡하고 힘들고, 작업의 한계가 금방 도래!
 - Matrix Multiplication
 - Matrix Concatenation
 - Inverse Matrix
 - Slicing Matrix
 - Matrix Decomposition
 -

A Better Way for Matrix Data

- NumPy Module in Python

NumPy Module 속의 ndarray라는 data type (class)를 OOP로 구현!

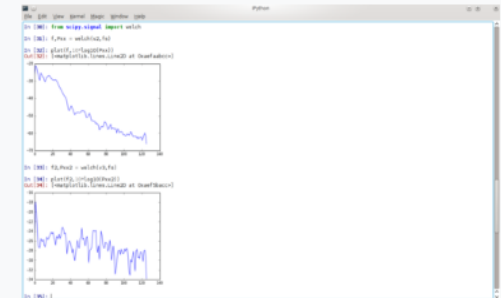
What is "numpy" Module?



Plot of $y=\sin(x)$ function, created with NumPy and Matplotlib libraries

Original author	<u>Travis Oliphant</u>
Developer	Community project
Initial release	As Numeric, 1995; as NumPy, 2006
Stable release	2.4.1 ^[1] / 10 January 2026; 54 days ago
Written in	Python, C
Operating system	Cross-platform
Type	Numerical analysis
License	BSD ^[2]
Website	numpy.org
Repository	github.com/numpy/numpy

SciPy



Power spectral density of ECG using SciPy

Original authors	<u>Travis Oliphant</u> , Pearu Peterson, Eric Jones
Developer	Community library project
Initial release	<u>Around 2001</u> ; 25 years ago
Stable release	1.17.1 ^[1] / 23 February 2026; 10 days ago
Written in	Python, Fortran, C, C++ ^[2]
Operating system	Cross-platform
Type	Technical computing
License	BSD-new license
Website	scipy.org
Repository	github.com/scipy/scipy

- **"NumPy" (Numeric Python):**
대규모 숫자 데이터의 배열(array)과 행렬(matrix)을 처리하고 조작하기 위한 기본적인 기능들을 제공하는 라이브러리
- **"SciPy" (Scientific Python) :**
NumPy의 기능을 확장하여 과학적 계산을 수행하기 위한 다양한 유용한 알고리즘들을 제공하는 라이브러리
 - Minimization, Fourier Transformation, Regression, and Other Applied Math Techniques
- Numpy and SciPy : open source add-on modules (not Python Standard Library)
- MatLab (1984년) and R (1993년)

<https://numpy.org>

[Install](#) [Documentation](#) [Learn](#) [Community](#) [About Us](#) [News](#) [Contribute](#) [English](#) ▾



The fundamental package for scientific computing with Python

LATEST RELEASE: NUMPY 2.4. VIEW ALL RELEASES

NumPy 2.4.0 released! [2025-12-20](#)

Powerful N-dimensional arrays

Fast and versatile, the NumPy vectorization, indexing, and broadcasting concepts are the de-facto standards of array computing today.

Numerical computing tools

NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines, Fourier transforms, and more.

Open source

Distributed under a liberal [BSD license](#), NumPy is developed and maintained [publicly on GitHub](#) by a vibrant, responsive, and diverse [community](#).

Interoperable

NumPy supports a wide range of hardware and computing platforms, and plays well with distributed, GPU, and sparse array libraries.

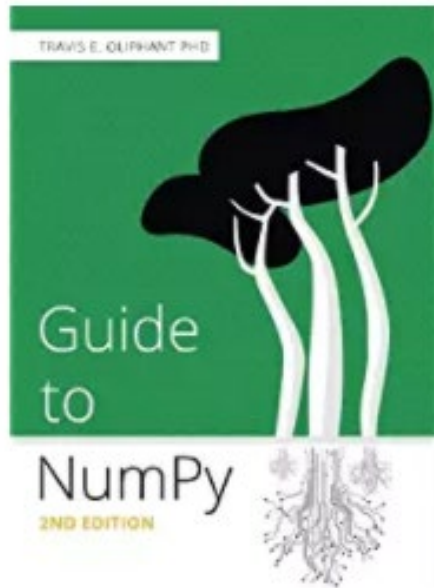
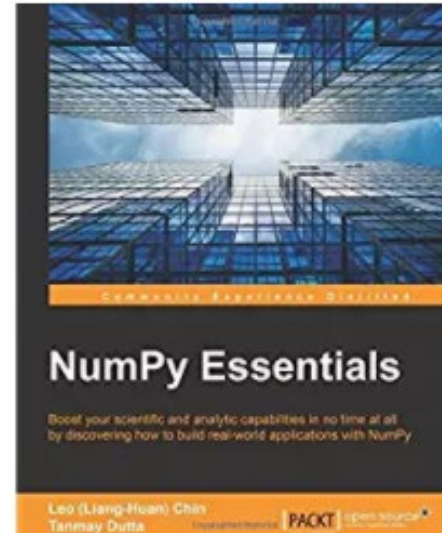
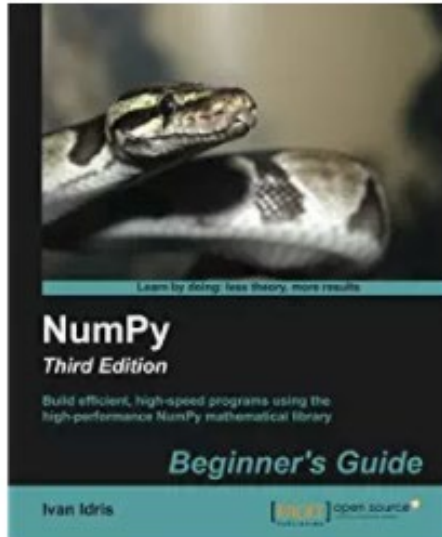
Performant

The core of NumPy is well-optimized C code. Enjoy the flexibility of Python with the speed of compiled code.

Easy to use

NumPy's high level syntax makes it accessible and productive for programmers from any background or experience level.

Many Numpy Books



Module (Library) 내부에는 무엇이 들어있나?

- **Classes (Data Types)**
 - class에 속한 functions
 - class에 속한 variables
- **Constant Variables:** 특정 상수값을 가진 변수들
- **Functions**
- **Submodules**

Numpy Module and ndarray Class

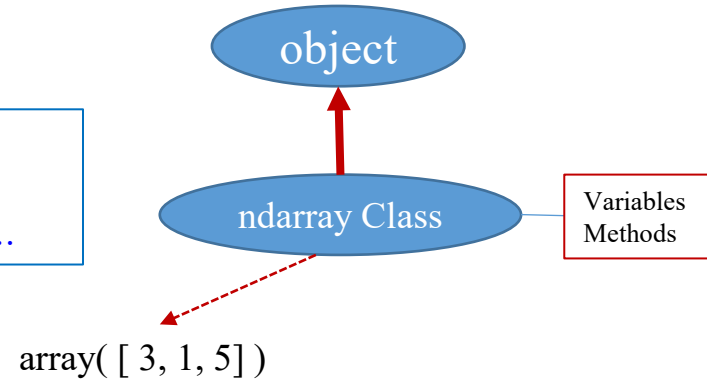
Numpy Module

- Class “**ndarray**”
 - Variables: shape, size, dtype,
 - Methods: **reshape()**, **flatten()**, **transpose()**, **mean()**,

- Constants
 - **numpy.pi**
 - **numpy.inf**
 - **numpy.nan**
 - **numpy.newaxis**
 -

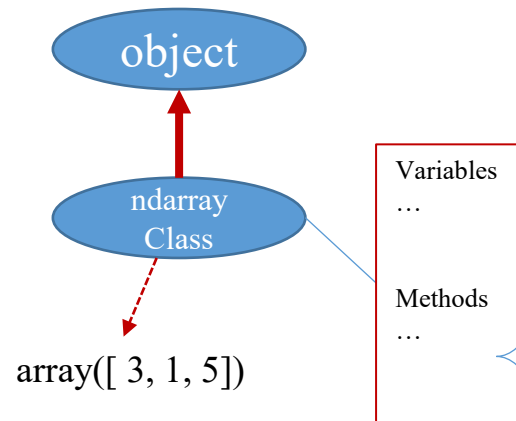
- Functions
 - **Functions for Array creation**
 - Functions for Numerical ranges
 - Functions for Changing array shape
 - Functions for Transpose operation
 - Functions for Concatenating arrays
 - ..

- Submodules
 - **Linalg submodule**
 - Distutils submodule
 -



Numpy 작업!!!: Narray object를 생성하고,
Numpy module에 있는 function을 사용하거나,
Narray class속에 있는 method를 사용한다

Ndarray Object를 생성하는 Numpy 모듈속에 있는 함수들



- array ()
- ones() : Return a new array setting values to one.
- zeros() : Return a new array setting values to zero.
- empty() : Return a new uninitialized array.
- full() : Return a new array of given shape filled with value
- ones_like() : Return an array of ones with shape and type of input.
- zeros_like() : Return an array of zeros with shape and type of input.
- empty_like() : Return an empty array with shape and type of input.
- full_like() : Return a new array with shape of input filled with value.
-

ndarray Object Creation : np.array() [1/3]

- An array can be created from a list:

```
import numpy as np
a = np.array([1, 2, 3, 4])
```

a

```
⇒ array([1, 2, 3, 4])
```

type(a)

```
⇒ numpy.ndarray
```

print(a)

```
⇒ [1 2 3 4]
```

a의 type이 뭐야?

Matrix 내부의 element
들의 data type

```
import numpy as np
a = np.array([1, 2, 3, 4], float)
```

a

```
⇒ array([1., 2., 3., 4.])
```

type(a)

```
⇒ numpy.ndarray
```

ndarray Object Creation : np.array() [2/3]

- Numpy arrays can be multidimensional

```
import numpy as np  
a = np.array([ [ 1, 2, 3 ], [4, 5, 6] ] )
```

a

⇒ array([[1, 2, 3],
[4, 5, 6]])

print(a)

[[1 2 3]
[4 5 6]]

a[0,0]

⇒ np.int64(1)

a[1,1]

⇒ np.int64(5)

print(a[0,0])

⇒ 1

$$\begin{vmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{vmatrix}$$

Python 2D List

▶ a = [[1, 2, 3],
[4, 5, 6]]

a[1][1]

⇒ 5

ndarray Object Creation: np.array() [3/3]

- `np.array(list)`: from a python list of numbers or strings as argument

```
>>> a= np.array(1,2,3,4)      # WRONG
>>> a= np.array([1,2,3,4])   # RIGHT
```

```
x = np.array( [1, 2, 3] )
```

```
type(x)
```

```
⇒ numpy.ndarray
```

x 의 type이 뭐야?

```
x.dtype
```

```
⇒ dtype('int64')
```

ndarray object x 를 구성하는
요소들의 data type이 뭐야?

```
x = np.array( [ 1, 2, 3.0] )
```

```
x.dtype
```

```
⇒ dtype('float64')
```

```
a = np.array( [ 'a', 'ab' ] )
```

```
a.dtype
```

```
⇒ dtype('<U2')
```

dtype parameter의 값

코드	의미	NumPy dtype	크기	값의 종류	정밀도/범위	
'i'	기본 정수	int32	4바이트	정수	약 $\pm 2.1 \times 10^9$	
'i8'	8바이트 정수	int64	8바이트	정수	약 $\pm 9 \times 10^{18}$	
'f'	기본 실수	float32	4바이트	실수	소수 약 7자리	
'f8'	8바이트 실수	float64	8바이트	실수	소수 약 15~16자리	

**** np.array() 에서 dtype parameter 값이 없는 상태에서**

- 요소들이 Integer 로만 구성되어 있으면 dtype('int64')
 - 요소들이 Float number가 1개라도 있으면 dtype('float64')
 - 요소들이 String 들로 구성되어 있으면 dtype(' <Ux ')
- ** 가장 긴 string의 길이 x

- Zero Matrix
- Matrix 덧셈의 항등원

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

- All-ones matrix = Ones Matrix
- 합·평균 등 계산에 사용

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \quad \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

- Identity Matrix = Unit Matrix
- 모든 값이 0이고 대각선방향 값만 1인 Matrix
- $I \times A = A \times I = A$
- 반드시 정방행렬 (square matrix) 이어야 함
- Matrix 곱셈의 항등원

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

ndarray Object Creation : np.zeros()

- np.zeros(array_size_tuple) for Zero Matrix. Default: filled with float 0.

```
import numpy as np
a = np.zeros( (3, 3) )
a
```

⇒ array([[0., 0., 0.],
[0., 0., 0.],
[0., 0., 0.]])

a.dtype

⇒ dtype('float64')

```
a = np.zeros( 5 )
a
```

⇒ array([0., 0., 0., 0., 0.]])

```
b = np.zeros( (2, 3) )
b
```

⇒ array([[0., 0., 0.],
[0., 0., 0.]])

array() 함수처럼 dtype을 명시하면 해당 data type들의 데이터를 가진 ndarray object를 생성

```
c = np.zeros( (5,2), dtype = 'i' )
c
```

⇒ array([[0, 0],
[0, 0],
[0, 0],
[0, 0],
[0, 0]], dtype=int32)

ndarray Object Creation : np.ones()

- 0이 아닌 1로 초기화된 ndarray object를 생성하려면 `ones()` 함수를 사용
- `np.ones( array_size_tuple )` → default: filled with (float) 1.

```
a = np.ones( (3, 3) )
```

```
a
```

3 X 3 matrix



```
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
```

```
a.dtype
```



```
dtype('float64')
```

```
e = np.ones( (2, 3, 4), dtype = 'i8' )
```

```
e
```

2 X 3 X 4 matrix

```
array([[[[1, 1, 1, 1],
         [1, 1, 1, 1],
         [1, 1, 1, 1]],
```

```
       [[1, 1, 1, 1],
        [1, 1, 1, 1],
        [1, 1, 1, 1]])])
```

```
e.dtype
```

```
dtype('int64')
```

i → dtype('int32')
i8 → dtype('int64')

ndarray Object Creation : np.ones-like() & zeros_like()

- `ones_like( )` : 다른 Matrix의 shape에 1로 구성된 All-Ones Matrix 를 생성
- `zeros_like( )` : 다른 Matrix의 shape에 0로 구성된 Zero Matrix 를 생성

```
b = np.array( [ [1, 2, 3], [4, 5, 6] ], float)
b
```

```
⇒ array([[1., 2., 3.],
        [4., 5., 6.]])
```

```
f = np.ones_like(b, dtype='f')
f
```

 2 X 3 matrix

```
⇒ array([[1., 1., 1.],
        [1., 1., 1.]], dtype=float32)
```

```
f = np.zeros_like(b, dtype='f')
f
```

```
⇒ array([[0., 0., 0.],
        [0., 0., 0.]], dtype=float32)
```

- `np.full( array_size_tuple , value )` : `np.ndarray` filled with certain value

```
a = np.full((3,3), 7)
a
→ array([[7, 7, 7],
         [7, 7, 7],
         [7, 7, 7]])
```

- `np.eye( array_size )` : `np.ndarray` for **Identity matrix** (square matrix에만 해당)

```
a = np.eye(3)
a
→ array([[1., 0., 0.],
         [0., 1., 0.],
         [0., 0., 1.]])
```

The identity matrix is **diagonal square matrix with ones**, defined such that $IX = XI = X$

ndarray Object Creation: np.arange()

- `np.arange( start_number, end_number, interval )`
 - generating numpy object from numbers in a range
- numpy의 `arange()`는 python의 `range()`와 유사

Python `range()`

```
r1 = range(10)
r1
→ range(0, 10)
list(r1)
→ [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

r2 = range(3, 21, 2)
r2
→ range(3, 21, 2)
list(r2)
→ [ 3,  5,  7,  9, 11, 13, 15, 17, 19]

r3 = range(0, 2, 0.3)
r3
→ range(0, 2, 0.3)
list(r3)
→ [0., 0.3, 0.6, 0.9, 1.2, 1.5, 1.8]
```

Numpy `arange()`

```
np.arange(10) # 0 .. n-1
→ array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

np.arange(3, 21, 2) # 시작, 끝(포함하지 않음), 단계
→ array([ 3,  5,  7,  9, 11, 13, 15, 17, 19])

np.arange(0, 2, 0.3) # it accepts float arguments
→ array([0. , 0.3, 0.6, 0.9, 1.2, 1.5, 1.8])
```

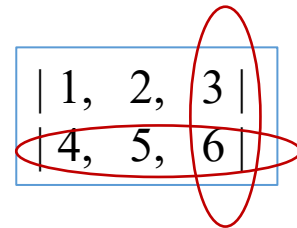
```
import numpy as np
```

```
a = np.array([[1, 2, 3], [4, 5, 6]], float)
```

row
column
`a[ 1, : ]`
→ `array([ 4., 5., 6.])`

`a[ :, 2 ]`
→ `array([ 3., 6.])`

`a[ -1: , -2: ]`
→ `array([[ 5., 6.]])`



A diagram showing a 2x3 matrix with elements 1, 2, 3 in the first row and 4, 5, 6 in the second row. A blue box highlights the entire matrix. A red oval highlights the second column (elements 2, 5, 6). A red arrow points from the text 'row' to the first row, and another red arrow points from the text 'column' to the second column.

1	2	3
4	5	6

Matrix a 는 그대로 있다!

Numpy Object의 일부분을 뽑아내기

[2/2]

```
import numpy as np
```

```
lst = [ [1, 2, 3],  
        [4, 5, 6],  
        [7, 8, 9] ]
```

```
arr = np.array(lst)
```

arr

```
→ array([[1, 2, 3],  
        [4, 5, 6],  
        [7, 8, 9]])
```

1	2	3
4	5	6
7	8	9

0~1 rows

0~1 columns

```
a = arr[ 0:2 , 0:2 ]
```

a

```
→ array([[1, 2],  
        [4, 5]])
```

```
print(a)
```

```
→ [[1 2]  
    [4 5]]
```

```
b = arr[ 1: , 1: ]
```

b

```
→ array([[5, 6],  
        [8, 9]])
```

```
print(b)
```

```
→ [[5 6]  
    [8 9]]
```

Numpy Object의 행(column)과 열(row)를 제거하기

```
import numpy as np
sample = np.array([ [1,2,3,4],
                    [5,6,7,8],
                    [3,3,3,3], ])
```

```
del_row1 = np.delete(sample, 1, axis=0) # 1번째 row의 모든 column에 대해서 값을 제거하라 → 1번째 row 제거
print(del_row1)
```

```
array([[1 2 3 4],
       [3 3 3 3]])
```

```
del_col1 = np.delete(sample, 1, axis=1) # 1번째 column의 모든 row에 대해서 값을 제거하라 → 1번째 column 제거
print(del_col1)
```

```
array([[1 3 4],
       [5 7 8],
       [3 3 3]])
```

Axes : axis의 복수 (= dimensions in numpy.ndarray object)

Numpy function 안에서 optional parameter 로

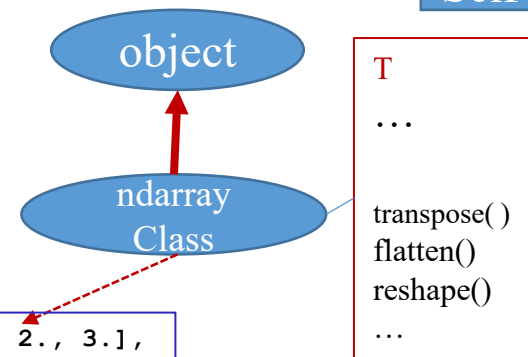
axis = 0 : 특정 row에 대해서 (= 모든 column에 대해서)

axis = 1 : 특정 column에 대해서 (= 모든 row에 대해서)

Transposing ndarray Object

```
>>> import numpy as np
```

- `transpose()`: 행과 열을 바꾸는 함수
- `T`: 행과 열을 바꾼 행렬을 보유하고 있는 attribute



```
array([[1., 2., 3.],
       [4., 5., 6.]])
```

a

```
a = np.array([[1, 2, 3], [4, 5, 6]], float)
```

a

```
→ array([[1., 2., 3.],
        [4., 5., 6.]])
```

a.transpose()

```
→ array([[1., 4.],
        [2., 5.],
        [3., 6.]])
```

- 2 X 3 Matrix를 3 X 2 Matrix로 transpose된 결과를 보여준다
- Matrix a는 그대로 있다!
- Matrix a를 변화시키고 싶으면 `a = a.transpose()`

attribute

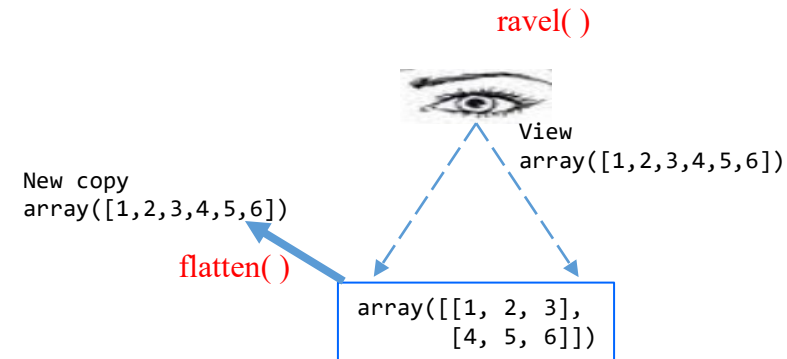
a.T

```
→ array([[1., 4.],
        [2., 5.],
        [3., 6.]])
```

Flattening ndarray Object

Flattening 2D np.ndarray into an 1D np.ndarray

- `flatten()` and `ravel()` looks similar
 - 2D Matrix, 3D Matrix 상관없이 전부 1D Matrix 로 변환
- `flatten()` returns a new copy while `ravel()` returns a view



```
a = np.array([[1, 2, 3],[4, 5, 6]])
```

```
a
→ array([[1, 2, 3],
        [4, 5, 6]])
```

```
b = a.flatten()
```

```
b
→ array([1, 2, 3, 4, 5, 6])
```

```
a[0, 0] = 100
```

```
a
→ array([[100, 2, 3],
        [4, 5, 6]])
```

```
b
→ array([1, 2, 3, 4, 5, 6])
```

```
a = np.array([[1, 2, 3],[4, 5, 6]])
```

```
c = a.ravel()
```

```
c
→ array([1, 2, 3, 4, 5, 6])
```

```
a[0, 0] = 100
```

```
a
→ array([[100, 2, 3],
        [4, 5, 6]])
```

```
c
→ array([100, 2, 3, 4, 5, 6])
```

* ravel: [망, 얽힌 사건]등을 풀어내다

Concatenating ndarray Objects

Concatenate 1D Array Objects

```
a = np.array([1,2], float)
b = np.array([3,4,5,6], float)
c = np.array([7,8,9], float)
np.concatenate( (a,b,c) )
→ array([1., 2., 3., 4., 5., 6., 7., 8., 9.])
```



Concatenate 2D Array Objects

```
a = np.array([[1,2], [3,4]])
```

```
b = np.array([[5,6]])
```

```
np.concatenate( (a,b) )
```

```
→ array([[1, 2],
        [3, 4],
        [5, 6]])
```

```
np.concatenate((a,b), axis=0)
```

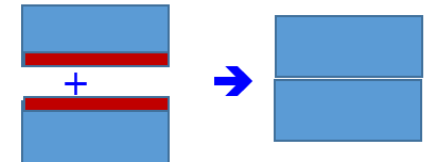
```
→ array([[1, 2],
        [3, 4],
        [5, 6]])
```

```
np.concatenate((a,b.T), axis=1)
```

```
→ array([[1, 2, 5],
        [3, 4, 6]])
```

axis = 0 가 default

a 의 모든 column에 대해서
내부에 concatenation 을 적용
→ matrix에서 row가 증가



a 의 모든 row에 대해서 내부에
concatenation 을 적용
→ matrix에서 column이 증가



Numpy function 안에서 optional parameter 로

axis = 0 : 각 column에 대해서

axis = 1 : 각 row에 대해서

Reshaping ndarray Object

```
a = np.arange(12)      # 1D Array
a
→ array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]) # 1X12 matrix

b = a.reshape(3, 4)   # 1X12 matrix a를 3 X 4 matrix 로 변환
b                       # a는 그대로 있다
→ array([[0, 1, 2, 3],
         [4, 5, 6, 7],
         [8, 9, 10, 11]])
```

```
a = np.array([[1, 2, 3],[4, 5, 6]])
a
→ array([[1, 2, 3],
         [4, 5, 6]])

a.reshape(3, 2)      # 3 X 2 matrix 로 변환

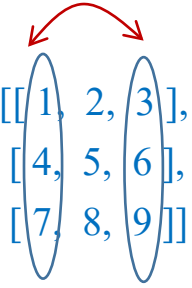
→ array([[1, 2],
         [3, 4],
         [5, 6]])

a                       # a는 그대로 있다
→ array([[1, 2, 3],
         [4, 5, 6]])
```

2D Numpy ndarray에서 서로 다른 column 2개를 교환

```
import numpy as np
```

```
original_array = np.array( [[1, 2, 3],  
                             [4, 5, 6],  
                             [7, 8, 9]])
```



```
# Matrix안에서 교환할 두 개의 column 인덱스 지정
```

```
column1_index = 0
```

```
column2_index = 2
```

```
# Matrix안에서 두 개의 column을 교환
```

```
original_array[ :, column1_index, column2_index ] = original_array[ :, column2_index, column1_index ]
```



```
# 결과 출력
```

```
print("Array with Exchanged Columns:")
```

```
print(original_array)
```

Array with Exchanged Columns:

[[3 2 1]

[6 5 4]

[9 8 7]]

2D Numpy ndarray에서 서로 다른 row 2개를 교환

```
import numpy as np
original_array = np.array([ [1, 2, 3],
                             [4, 5, 6],
                             [7, 8, 9] ])
# Matrix안에서 교환할 두 개의 row 인덱스 지정
row1_index = 0
row2_index = 2
# Matrix안에서 두 개의 row을 교환
original_array[row1_index, row2_index, :] = original_array[row2_index, row1_index, :]
# 결과 출력
print("Array with Exchanged Rows:")
print(original_array)
```

Array with Exchanged Rows:

```
[[7 8 9]
 [4 5 6]
 [1 2 3]]
```

Reshaping ndarray Object with 'newaxis'

1D ndarray 생성

```
x = np.arange(5)
x
→ array([0, 1, 2, 3, 4])
```

```
x.reshape(5,1)
→ array([[0],
        [1],
        [2],
        [3],
        [4]])
```

1D ndarray 를 5x1 2D ndarray로 변환

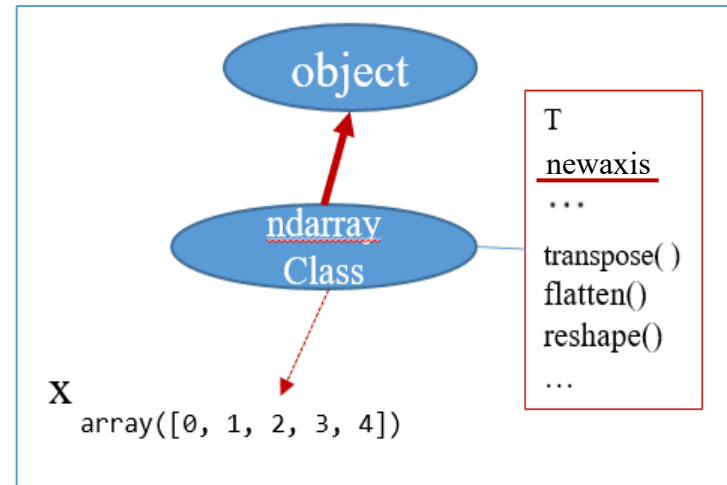
1D ndarray 를 1x5 2D ndarray로 변환

```
x.reshape(1, 5)
→ array([[0, 1, 2, 3, 4]])
```

```
x[:, np.newaxis]
→ array([[0],
        [1],
        [2],
        [3],
        [4]])
```

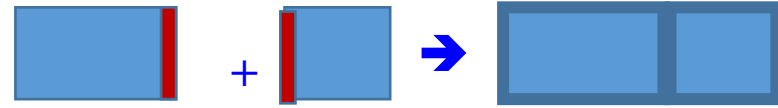
np module 속에
있는 variable

```
np.newaxis
-- 새로운 axis를 추가
-- 1D ndarray 를 5x1 2D ndarray로 변환
```

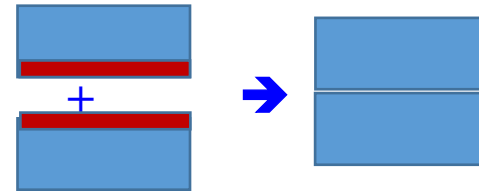


Various Stacking with Matrices (in np module)

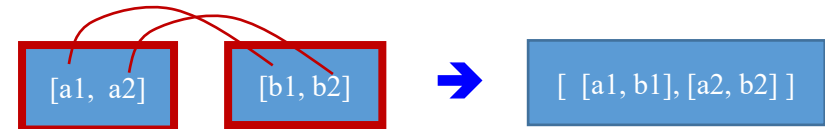
- `np.hstack()` : horizontal stacking
 - row숫자가 같은 matrix들에만 적용



- `np.vstack()` : vertical stacking
 - column숫자가 같은 matrix들에만 적용



- `np.dstack()` : dimensional stacking
 - Shape이 같은 2개의 matrix들에 적용
 - Corresponding Element 2개를 list로 만들어서 배치



- `np.stack()` : general stacking
 - Shape이 같은 2개의 matrix를 list속에 배치
 - Horizontal 배치형
 - Row 확장형

Horizontal Stacking : np.hstack()

```
a1 = np.ones((2,3))
a1
→ array([[1., 1., 1.],
         [1., 1., 1.]])
```

```
a2 = np.zeros((2,2))
a2
→ array([[0., 0.],
         [0., 0.]])
```

```
np.hstack([a1, a2])
```

```
→ array([[1., 1., 1., 0., 0.],
         [1., 1., 1., 0., 0.]])
```

Horizontal Stacking



결과물의 dimension 그대로
 결과물의 column 수는 증가
 결과물의 row 수는 그대로

- `hstack()`을 수행할때에 참여하는 **matrix의 row의 갯수들은 반드시 같아야 한다**
- Row 값이 같은 여러 개의 matrix 를 `hstack()` 하는것도 가능하다



Remember?

```
np.concatenate((a1, a2), axis=1)
→ array([[1., 1., 1., 0., 0.],
         [1., 1., 1., 0., 0.]])
```

np.hstack()으로 2D Numpy Object에 Column 추가

self study

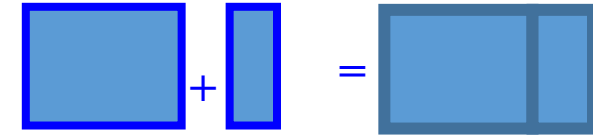
```
original_array = np.array([ [1, 2, 3],  
                             [4, 5, 6],  
                             [7, 8, 9]])
```

```
new_column = np.array( [[10], [11], [12]] ) ← 2D ndarray
```

```
new_array = np.hstack([original_array, new_column] )
```

```
print("\n New Array with Added Column:")
```

```
print(new_array)
```



New Array with Added Column:

```
[[ 1  2  3 10]  
 [ 4  5  6 11]  
 [ 7  8  9 12]]
```

```
original_array = np.array([ [1, 2, 3],  
                             [4, 5, 6],  
                             [7, 8, 9]])
```

```
new_column = np.array( [ 10, 11, 12] ) ← 1D ndarray
```

```
new_array = np.column_stack([ original_array, new_column ] )
```

```
print("\n New Array with Added Column:")
```

```
print(new_array)
```

New Array with Added Column:

```
[[ 1  2  3 10]  
 [ 4  5  6 11]  
 [ 7  8  9 12]]
```

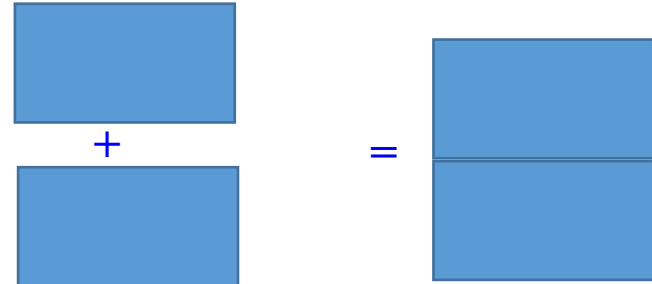
Vertical Stacking : np.vstack()

```
b1 = np.ones((2,3))
b1
→ array([[1., 1., 1.],
         [1., 1., 1.]])

b2 = np.zeros((3,3))
b2
→ array([[0., 0., 0.],
         [0., 0., 0.],
         [0., 0., 0.]])
```

```
np.vstack([b1, b2])
→ array([[1., 1., 1.],
         [1., 1., 1.],
         [0., 0., 0.],
         [0., 0., 0.],
         [0., 0., 0.]])
```

Vertical Stacking



결과물의 dimension 그대로
결과물의 column 수는 그대로
결과물의 row 수는 증가

- `vstack()`을 수행할때에 참여하는 matrix의 column의 갯수들은 반듯이 같아야 한다
- Column 값이 같은 여러 개의 matrix를 `vstack()` 하는것도 가능하다

Remember?

```
np.concatenate((b1, b2), axis=0)
→ array([[1., 1., 1.],
         [1., 1., 1.],
         [0., 0., 0.],
         [0., 0., 0.],
         [0., 0., 0.]])
```

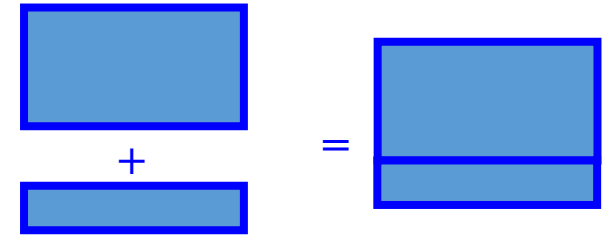
np.vstack()으로 2D Numpy Object에 Row 추가

```
original_array = np.array([ [1, 2, 3],
                             [4, 5, 6],
                             [7, 8, 9]])

new_row= np.array( [ 10, 11, 12] )

new_array = np.vstack( [ original_array, new_row ] )

print("\n New Array with Added Column:")
print(new_array)
```



```
original_array = np.array([ [1, 2, 3],
                             [4, 5, 6],
                             [7, 8, 9]])

new_row= np.array( [ 10, 11, 12] )

new_array = np.row_stack( [ original_array, new_row ] )

print("\n New Array with Added Column:")
print(new_array)
```

New Array with Added Column:

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

New Array with Added Column:

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

Dimensional Stacking: dstack()

```
c1 = np.ones((3,4))
c1
→ array([[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]])
```

```
c2 = np.zeros((3,4))
c2
→ array([[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]])
```

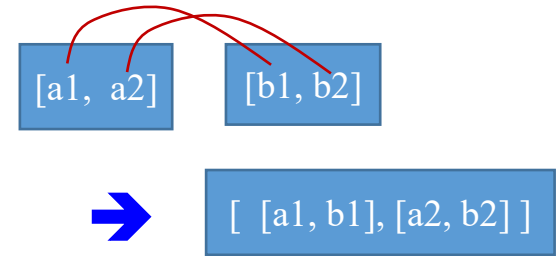
C1의 element와 C2의 corresponding element 를 새로운 list로 만듦

```
np.dstack( [c1, c2] )
→ array([[ 1., 0.],
         [1., 0.],
         [1., 0.],
         [1., 0.]])
```

```
[ [1., 0.],
  [1., 0.],
  [1., 0.],
  [1., 0.]]
```

```
(np.dstack([c1, c2])).shape
```

```
→ (3, 4, 2)
```



Dimensional Stacking
Depth Stacking

결과물의 dimension이 증가
결과물의 column 수는 그대로
결과물의 row 수는 그대로

○

- dstack()을 수행할때에 참여하는 matrix들의 column 갯수와 row 갯수들은 반드시 각각 같아야 한다 (same shape)
- column 값, row값이 같은 여러 개의 matrix 를 dstack() 하는것도 가능하다
- 가장 안쪽의 원소가 list가 되므로 결과 matrix 의 dimension이 증가한다.

Horizontal 배치형 General Stacking

[.....]

[.....]



[.....]



[.....]



Default: axis = 0

모든 column에 대해서

```
c1 = np.ones((3,4))
```

```
c1
```

```
→ array([[1., 1., 1., 1.],  
         [1., 1., 1., 1.],  
         [1., 1., 1., 1.]])
```

```
c2 = np.zeros((3,4))
```

```
c2
```

```
→ array([[0., 0., 0., 0.],  
         [0., 0., 0., 0.],  
         [0., 0., 0., 0.]])
```

```
c = np.stack([c1, c2], axis=0)
```

```
c
```

```
→ array([[1., 1., 1., 1.],  
         [1., 1., 1., 1.],  
         [1., 1., 1., 1.]])
```

```
[[0., 0., 0., 0.],  
 [0., 0., 0., 0.],  
 [0., 0., 0., 0.]])
```

```
c.shape
```

```
→ (2, 3, 4)
```

c1, c2를 새로운 list 속에 나열
(즉, Dimension이 증가)

결과물의 dimension이 확장

- stack()을 수행할때에 참여하는 matrix들의 column 값과 row 값들은 반듯이 같아야 한다 (same shape)
- column 값, row값이 같은 여러 개의 matrix 를 stack() 하는것도 가능하다

General Stacking: np.stack() [2/2]

Row 확장형 General Stacking

모든 row에 대해서

```
c1 = np.ones((3,4))
c1
→ array([[1., 1., 1., 1.],
          [1., 1., 1., 1.],
          [1., 1., 1., 1.]])

c2 = np.zeros((3,4))
c2
→ array([[0., 0., 0., 0.],
          [0., 0., 0., 0.],
          [0., 0., 0., 0.]])
```

```
c = np.stack([c1, c2], axis=1)
c
→ array([[1., 1., 1., 1.],
          [0., 0., 0., 0.]],
          [[1., 1., 1., 1.],
          [0., 0., 0., 0.]],
          [[1., 1., 1., 1.],
          [0., 0., 0., 0.]])

c.shape
→ (3, 2, 4)
```

c1의 모든 row에 대해서 c2 row를 확장시키고 Dimension을 증가시킨것

C1의 row와 C2의 corresponding row 를 새로운 list로 만들

- stack()을 수행할때에 참여하는 matrix들의 column 값과 row 값들은 반듯이 같아야 한다 (same shape)

Mathematics for ndarray Objects [1/4]

```
a = np.array([1, 2, 3], float)
b = np.array([5, 2, 6], float)
```

a + b ← • Matrix 들의 대응요소끼리 더하기
→ array([6., 4., 9.])

a - b ← • Matrix 들의 대응요소끼리 빼기
→ array([-4., 0., -3.])

a * b ← • Matrix 들의 대응요소끼리 곱하기
• Matrix multiplication과는 혼동하지 말아야 함!
→ array([5., 4., 18.])

b / a ← • Matrix 들의 대응요소끼리 나누기
→ array([5., 1., 2.])

a % b ← • Matrix 들의 대응요소끼리
나눈후 나머지를 반환 (modulo)
→ array([1., 0., 3.])

b ** a ← • Matrix 들의 대응요소끼리 지수승
→ array([5., 4., 216.])

- 같은 shape의 numpy objects에 4칙연산을 하면 대응요소 (**corresponding element**)들간에 연산이 적용된다!
- 그러나 a 혹은 b 자리에 constant를 넣어도OK!

Mathematics for ndarray Objects [2/4]

```
a = np.array([1,2,3], float)
b = np.array([5,2,6], float)
```

a + b

→ `array([5., 7., 9.])`

```
b2 = np.array([4,5,6,7], float)
```

Shape 이 다른 Matrix들에
4칙 연산이 작동할수 없다!

a + b2

→ `Traceback (most recent call last)`

`<ipython-input-72-1040507f52d2> in <module>`

`----> 1 a+b2`

`ValueError: operands could not be broadcast together with shapes (3,) (4,)`

b2 / a

→ `Traceback (most recent call last)`

`<ipython-input-73-ec1979e8661c> in <module>`

`----> 1 b2 / a`

`ValueError: operands could not be broadcast together with shapes (4,) (3,)`

b2 ** a

→ `Traceback (most recent call last)`

`<ipython-input-74-9205f2425e39> in <module>`

`----> 1 b2 ** a`

`ValueError: operands could not be broadcast together with shapes (4,) (3,)`

Mathematics for ndarray Objects [3/4]

- ‘*’ operation in python_lists vs. ‘*’ operation in np.ndarrays

- python_list * integer : Repetition of the whole list

```
a = [1,2,3,4]
```

```
a * 2
```

```
→ [1, 2, 3, 4, 1, 2, 3, 4]
```

- np.ndarray * integer : multiply integer to every element in np.ndarray object

```
a = np.array([1, 2, 3, 4])
```

```
a
```

```
→ array([1, 2, 3, 4])
```

```
a * 2
```

```
→ array([2, 4, 6, 8])
```

Mathematics for ndarray Objects [4/4]

- ‘+’ operation in python_lists vs. ‘+’ operation in np.ndarrays

- python_list + python_list : Concatenating two python_lists into one python_list

```
a = [1,2,3,4]
b = [1,2,3,4]
```

a + b

→ [1, 2, 3, 4, 1, 2, 3, 4]

```
a = [ [1,2], [3,4] ]
b = [ [5, 6], 100 ]
```

a + b

→ [[1, 2], [3, 4], [5, 6], 100]

- np.ndarray + np.ndarray : Pairwise + operation between two np.ndarrays

```
npa = np.array([1, 2, 3, 4])
npb = np.array([1, 2, 3, 4])
```

npa + npb

→ array([2, 4, 6, 8])

```
anp = np.array([[1, 2, 3],
                 [4, 5, 6],
                 [7, 8, 9]])
bnp = np.array([[1, 2, 3],
                 [4, 5, 6],
                 [7, 8, 9]])
```

anp + bnp

→ array([[2, 4, 6],
 [8, 10, 12],
 [14, 16, 18]])

Broadcasting in Numpy Mathematics

```
import numpy as np

A = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])

b = np.array([10,20,30])

C = A + b
print(C)
```

Broadcasting 과정

NumPy는 b를 아래처럼 확장하여 계산합니다.

```
[10 20 30]
[10 20 30]
[10 20 30]
```

결과

```
[[11 22 33]
 [14 25 36]
 [17 28 39]]
```

Numpy Matrix Operations in Numpy.linalg (Linear Algebra = Matrix Algebra)

- Matrix Multiplication
- Determinant of Matrix
- Inverse Matrix
- LU Decomposition of Matrix
- Eigen Value & Eigen Vector of Matrix
- SVD Decomposition of Matrix

최소한의
선형 대수
지식

Statistics in Numpy

No	함수	기능 (의미)
1	<code>np.mean()</code>	평균(average) 계산
2	<code>np.median()</code>	중앙값(median) 계산
3	<code>np.std()</code>	표준편차(standard deviation)
4	<code>np.var()</code>	분산(variance)
5	<code>np.min()</code>	최소값(minimum)
6	<code>np.max()</code>	최대값(maximum)
7	<code>np.sum()</code>	전체 합(sum)
8	<code>np.percentile()</code>	퍼센타일 값 계산
9	<code>np.quantile()</code>	분위수 계산
10	<code>np.corrcoef()</code>	상관계수(correlation coefficient)

최소한의
기초 통계
지식

AI Coding을 위해서 알아야하는 Numpy Details!

1. **Vectorization (벡터화)**: Matrix연산을 Python 반복문(for) 대신 Numpy 전체에 연산을 적용하여 쉽게 coding하고, 연산 속도를 비약적으로 향상시킴
2. **Broadcasting (브로드캐스팅)**: 모양(Shape)이 다른 Matrix 간의 연산 시, Numpy 특정 규칙에 따라 계산을 가능하게 함
3. **Slicing & Indexing**: Matrix에서 Numpy 슬라이싱과 인덱싱으로 새로운 배열을 만들어냄
4. **Shape Manipulation (차원 변형)**: Numpy 연산자 (reshape, flatten, transpose(T)..) 등을 통해 Matrix의 구조를 자유자재로 변경
5. **Data Types (dtype)**: Numpy data type에서 Matrix의 요소들의 데이터 타입(float32와 int64 등)을 명시적으로 관리
6. **Linear Algebra (선형대수)**: Numpy의 선형대수 연산자 (행렬 곱셈, 벡터 내적) 등을 수월하게 구사하기
7. **Aggregation (집계 함수)**: Numpy 통계함수 (np.mean, np.std, np.argmax 등)들을 통해 Matrix의 통계적 특성을 파악하거나 모델의 예측 클래스를 추출